Universidad Autónoma de Madrid

Escuela Politécnica Superior

26|27

Bachelor thesis

Cookies and their Effect on Privacy in Web Browsing



Milenka Ivanovic Marredo

Escuela Politécnica Superior
Universidad Autónoma de Madrid
C\Francisco Tomás y Valiente nº 11

Campus Internacional excelencia UAM CSIC

www.uam.es

UNIVERSIDAD AUTÓNOMA DE MADRID
ESCUELA POLITÉCNICA SUPERIOR





Bachelor as Ingeniería Informática

BACHELOR THESIS

Cookies and their Effect on Privacy in Web Browsing

Author: Milenka Ivanovic Marredo
Advisor: Daniel Perdices Burrero

septiembre 2026

All rights reserved.
No reproduction in any form of this book, in whole or in part
(except for brief quotation in critical articles or reviews),
may be made without written authorization from the publisher.

© May 20th, 2026 by UNIVERSIDAD AUTÓNOMA DE MADRID
Francisco Tomás y Valiente, n° 1
Madrid, 28049
Spain

Milenka Ivanovic Marredo
Cookies and their Effect on Privacy in Web Browsing

Milenka Ivanovic Marredo
C\ Francisco Tomás y Valiente Nº 11

PRINTED IN SPAIN

To my family, for their endless love, patience, and support throughout this journey. Especially to my dad, whose encouragement and trust in me gave me the strength to keep going even during the hardest moments. To my brothers, Branko and Milko, for all the laughter, support, and unforgettable moments along the way. And, of course, to Miguelón, my cat, for his companionship and joy.

“It always seems impossible until it’s done.”

Nelson Mandela

Agradecimientos

I would like to express my deepest gratitude to my family for always being by my side and for their unconditional support throughout this journey. Their love, encouragement, patience, and constant belief in me have been my greatest source of strength, especially during the most challenging moments of this degree.

I am especially grateful to my supervisor, Daniel Perdices, for his patience, guidance, and wisdom. His support, thoughtful advice, and confidence in my work have been essential to the development of this project and have helped me grow both academically and personally.

Finally, I would like to sincerely thank all my professors throughout this degree. Their dedication, passion for teaching, and commitment to their work have had a profound impact on my education. Thanks to them, I not only acquired knowledge, but also discovered a genuine interest and enjoyment in this field, which has strongly shaped my academic path.

Resumen

La continua evolución de las tecnologías web y la adopción generalizada de protocolos de comunicación cifrada han transformado significativamente el campo del análisis de tráfico de red. Los enfoques tradicionales basados en *Deep Packet Inspection* (DPI), que dependían de la inspección del contenido de los paquetes y de la información de la capa de aplicación, se han vuelto cada vez menos efectivos debido al despliegue de HTTPS, TLS 1.3, QUIC, DNS over HTTPS (DoH) y tecnologías VPN. Como resultado, las técnicas modernas de análisis de tráfico deben basarse en metadatos y patrones de comportamiento del tráfico en lugar de en la inspección directa de los paquetes.

Este trabajo estudia el problema de la identificación de navegación web a partir de tráfico cifrado bajo diferentes escenarios de consentimiento de cookies. El principal objetivo del trabajo es analizar si el comportamiento de navegación y los sitios web visitados pueden seguir siendo inferidos a partir del tráfico de red cifrado, a pesar de las limitaciones introducidas por los mecanismos modernos de privacidad y las regulaciones de consentimiento de cookies. En particular, el trabajo evalúa cómo aceptar o rechazar cookies afecta al tráfico observable generado durante las sesiones de navegación web y al rendimiento de modelos de clasificación basados en aprendizaje automático.

Para abordar este problema, se diseñó e implementó un framework automatizado de recolección de tráfico capaz de capturar sesiones de navegación en un entorno controlado. El framework permite la generación y recopilación automática de tráfico web cifrado asociado a diferentes configuraciones de consentimiento, incluyendo escenarios de aceptación y rechazo de cookies. Posteriormente, el tráfico capturado fue procesado y transformado en representaciones estructuradas adecuadas para su análisis mediante técnicas de aprendizaje automático. A lo largo del trabajo se evaluaron diferentes etapas de preprocesamiento y extracción de características del tráfico. Se analizaron distintas representaciones basadas en ocurrencias de dominios y metadatos del tráfico con el objetivo de estudiar la escalabilidad y complejidad del problema de clasificación.

Este trabajo también investiga el impacto de incrementar el número de clases objetivo en la tarea de clasificación. Los resultados experimentales demuestran que la precisión de clasificación disminuye progresivamente a medida que aumenta el número de dominios considerados, revelando importantes limitaciones de escalabilidad en los sistemas de clasificación de tráfico cifrado.

Los resultados obtenidos muestran que el tráfico cifrado todavía contiene patrones de comportamiento significativos que pueden explotarse para inferir la actividad de navegación. Sin embargo, también demuestran que la creciente complejidad de los ecosistemas web modernos, junto con la presencia de servicios de infraestructura compartida, recursos de terceros y comportamientos de navegación heterogéneos, introduce desafíos significativos para lograr una clasificación de tráfico precisa y escalable.

Palabras clave

Aceptar vs Rechazar, Dominios Elefante, Dominios Ratón, Crecimiento del Vocabulario, Metadatos de Tráfico, Complejidad de Clasificación.

Abstract

The continuous evolution of web technologies and the widespread adoption of encrypted communication protocols have significantly transformed the field of network traffic analysis. Traditional approaches based on Deep Packet Inspection (DPI), which relied on inspecting packet payloads and application-layer information, have become increasingly ineffective due to the deployment of HTTPS, TLS 1.3, QUIC, DNS over HTTPS (DoH), and VPN technologies. As a result, modern traffic analysis techniques must rely on traffic metadata and behavioral patterns rather than direct packet inspection.

This thesis studies the problem of web browsing identification from encrypted traffic under different cookie consent scenarios. The main objective of the work is to analyze whether browsing behavior and visited websites can still be inferred from encrypted network traffic despite the limitations introduced by modern privacy mechanisms and cookie consent regulations. In particular, the work evaluates how accepting or rejecting cookies affects the observable traffic generated during web browsing sessions and the performance of machine learning classification models.

To address this problem, an automated traffic collection framework was designed and implemented to capture browsing sessions in a controlled environment. The framework allows the automatic generation and collection of encrypted web traffic associated with different consent configurations, including cookie acceptance and rejection scenarios. The captured traffic was then processed and transformed into structured representations suitable for machine learning analysis.

Several stages of traffic preprocessing and feature extraction were evaluated throughout the work. Different traffic representations based on domain occurrences and traffic metadata were analyzed in order to study the scalability and complexity of the classification problem. The thesis also investigates the impact of increasing the number of target classes in the classification task.

Experimental results demonstrate that classification accuracy progressively decreases as the number of considered domains grows, revealing important scalability limitations in encrypted traffic classification systems.

The obtained results show that encrypted traffic still contains meaningful behavioral patterns that can be exploited to infer browsing activity. However, they also demonstrate that the increasing complexity of modern web ecosystems, together with the presence of shared infrastructure services, third-party resources, and heterogeneous browsing behavior, introduces significant challenges for accurate and scalable traffic classification.

Keywords

Accept vs Deny, Elephant Domains, Mice Domains, Vocabulary Growth, Traffic Metadata, Classification Complexity.

Table of Contents

1 Introduction **1**

1.1 Motivation . . . 1

1.2 Objectives . . . 2

1.3 Document structure . . . 2

2 State of the Art **5**

2.1 Web Traffic Data Collection and Browsing Simulation . . . 5

2.2 Web Browsing and Application Identification . . . 7

2.3 Cookies and Web Session Management . . . 9

3 Methodology **13**

3.1 Web browsing data . . . 16

3.1.1 Cookie Consent Interaction Methodology . . . 16

3.1.2 Network Packet Processing . . . 17

3.2 Data Analysis and Feature Extraction . . . 19

3.2.1 Embeddings . . . 19

3.2.2 Machine Learning Model . . . 20

4 Evaluation and Results **23**

4.1 Elephants and Mice . . . 23

4.1.1 Traffic Distribution and Domain Persistence . . . 23

4.1.2 The Vocabulary Problem . . . 25

4.2 Accept vs Deny vs No action . . . 27

4.3 Impact of Classification Complexity . . . 30

5 Conclusion and Future Work **33**

5.1 Conclusion . . . 33

5.2 Future work . . . 34

Bibliography **37**

Appendices **39**

A Alternative Client-Side Storage Mechanisms **41**

A.1 Local Storage . . . 41

A.2 Session Storage . . . 41

A.3 IndexedDB . . . 42

A.4 Privacy Implications . . . 42

B *Accept Only Essentials* Consent Configurations **43**

Lists

List of figures

3.1 *Overview of the methodology pipeline from data collection to model evaluation.* 13

4.1 *Session-domain presence heatmap showing the difference between persistent elephant domains and sparse mice domains across browsing sessions.* 24

4.2 *Long-tail distribution of domain frequencies showing that a small subset of domains dominates the traffic while most domains appear infrequently.* 25

4.3 *Comparison between the vocabulary growth of Count Vectorization and the fixed-dimensional representation of Hashing Vectorization.* 27

4.4 *Dominance of elephant domains within the feature space generated from browsing sessions.* 28

4.5 *Classification accuracy for the Accept and Deny scenarios under different Top-N domain configurations.* 29

4.6 *Difference in classification accuracy between the Accept and Deny scenarios for different Top-N domain configurations.* 30

4.7 *Classification accuracy degradation as the number of classes increases.* 31

4.8 *Reduction in the number of samples per class as the complexity of the classification problem increases.* 31

1

Introduction

1.1. Motivation

In contemporary network environments, user privacy has become a critical requirement. The rapid expansion of digital services, cloud infrastructures, and interconnected devices has led to an unprecedented increase in the volume and sensitivity of data transmitted across networks. As a result, protecting user information is no longer optional but a fundamental aspect of system design and operation.

Current network ecosystems must guarantee user privacy to reduce exposure to fraud and phishing-based attacks. Cybercriminals continuously exploit vulnerabilities in communication systems to deceive users, steal credentials, and gain unauthorized access to sensitive information. These attacks are becoming increasingly sophisticated, often blending seamlessly with legitimate traffic, which makes their detection significantly more challenging.

At the same time, with the exponential growth of network data, automated and intelligent systems are required to detect anomalies associated with fraudulent behavior. Traditional rule-based approaches are no longer sufficient to cope with the scale, complexity, and dynamic nature of modern cyber threats. Instead, advanced data-driven techniques, particularly those based on machine learning, have emerged as promising solutions to analyze large datasets and identify subtle patterns indicative of malicious activity.

Improving early detection of fraud and phishing contributes directly to enhancing user trust and security in digital infrastructures. Early identification of threats allows for timely mitigation, reducing both financial losses and the potential compromise of sensitive data. Moreover, increasing user confidence in digital platforms is essential for the continued growth and adoption of online services.

However, balancing effective threat detection with user privacy preservation remains a key challenge in modern network security, requiring innovative analytical solutions. While deep inspection of network data can improve detection accuracy, it may also introduce privacy concerns. Therefore, it is crucial to develop approaches that achieve high detection performance without compromising the confidentiality of user information.

1.2. Objectives

The main objective of this thesis is to analyze and characterize web browsing behavior through the study of network traffic generated during browsing sessions, with special focus on cookies, third-party services, and tracking-related communications. In order to achieve this goal, the work is structured around the following objectives:

1.– Design and implementation of a controlled environment capable of automatically capturing web browsing traffic under different cookie consent scenarios, including cookie acceptance, rejection, and situations where no interaction is performed.

2.– Analysis of the generated network traffic to identify patterns related to DNS queries, TLS SNI values, server IP communications, and interactions with external services and tracking infrastructures.

3.– Study of the domain distribution present in the captured traffic, distinguishing between highly recurrent domains (*elephants*) and low-frequency domains (*mice*), together with an evaluation of their impact on traffic representation and feature extraction.

4.– Analysis of the influence of cookie consent decisions on the resulting network traffic in order to determine whether accepting or rejecting cookies significantly modifies the observable communication patterns.

5.– Evaluation of different embedding and feature representation techniques, including vocabulary-based approaches and hashing-based representations, to study their scalability and suitability for encrypted traffic analysis.

6.– Training and comparison of machine learning models capable of identifying visited domains from encrypted network traffic, evaluating how the complexity of the classification problem evolves as the number of domains increases.

7.– Finally, assessment of whether encrypted traffic metadata alone provides sufficient information to infer browsing activity, even in scenarios where cookies are rejected or explicit tracking mechanisms are limited.

1.3. Document structure

The structure of this document is as follows. Chapter 2 presents an overview of existing solutions for web browsing privacy. Chapter 3 describes the implementation followed throughout this project, including the design decisions, implementation process, and experimental setup used to develop the proposed solution. Subsequently, Chapter 4 presents and evaluates the outcomes obtained from the implemented system. Finally, Chapter 5 summarizes the main contributions of this thesis, discusses the extent to which the proposed objectives were achieved, and outlines potential future lines of work.

The bibliography compiles all the references and sources consulted throughout the development of this work, while the appendix includes additional material and complementary information that support the content presented in the main chapters.

2

State of the Art

2.1. Web Traffic Data Collection and Browsing Simulation

A significant challenge in web privacy research is obtaining large-scale and reproducible browsing data without relying exclusively on real users. For this reason, many recent works have adopted automated web navigation frameworks to simulate realistic browsing activity and generate network traces. Instead of manually collecting user traffic, researchers increasingly rely on automated systems capable of visiting websites, interacting with webpages, and recording the resulting network behavior.

In the study *Web browsing privacy in the deep learning era: Beyond VPNs and encryption* [1] developed an automated browsing infrastructure capable of generating more than 340 GB of encrypted traffic traces by repeatedly visiting websites from the Alexa Top 500 ranking. The authors used browser automation techniques to systematically navigate websites and capture the resulting encrypted traffic flows, allowing them to construct a large-scale and reproducible dataset under controlled experimental conditions. This approach significantly reduced the logistical and ethical difficulties associated with collecting real user browsing traffic while still enabling realistic traffic generation.

The Alexa Top Sites ranking, historically known as the Alexa Top Million, was one of the most widely used popularity rankings in Internet measurement and web privacy research. Alexa Internet, a subsidiary of Amazon, generated these rankings using browsing information collected from browser extensions, tracking services, and websites integrating Alexa analytics tools. The ranking estimated website popularity based on metrics such as unique visitors and page views over a rolling time window. Due to its public availability and large scale, the Alexa ranking became a standard benchmark for selecting representative websites in studies related to web measurement, cybersecurity, and online privacy

However, the reliability and representativeness of website popularity rankings have also been critically analyzed in the literature. In the study *Toppling Top Lists: Evaluating the Accuracy of Popular Website Lists* [2] compared several popularity rankings, including Alexa, Cisco Umbrella, Majestic Million, Tranco, and the Chrome User Experience Report (CrUX). The authors showed that different rankings often produce significantly different domain selections due to the distinct methodologies used to esti-

mate popularity, such as DNS activity, browser telemetry, backlinks, or user navigation statistics. Their results highlighted that popularity rankings may contain biases and inconsistencies, which can directly affect the construction of web measurement datasets and experimental reproducibility.

An additional large-scale perspective on browsing behavior and web traffic collection was presented by Ruth et al. in the study *A World Wide View of Browsing the World Wide Web* [3]. In this work, the authors analyzed browsing activity collected from Google Chrome telemetry in order to better understand how users interact with the modern web ecosystem at a global scale. Unlike studies focused on controlled browsing automation, this work relied on real-world browsing data obtained from millions of users, providing a more representative view of actual web navigation patterns.

The dataset analyzed in the study contained anonymized browsing information collected from users who had consented to share telemetry data through Chrome. The authors studied several aspects of browsing behavior, including website popularity distribution, browsing session characteristics, domain diversity, content concentration, and traffic distribution across web services. One of the most relevant findings was the strong concentration of browsing activity around a relatively small subset of highly popular domains, while the majority of websites received comparatively little traffic. This observation is particularly relevant in the context of traffic classification problems, as it naturally creates class imbalance issues where a small number of domains dominate the dataset.

The study also highlighted the complexity of modern web browsing sessions. A single webpage visit often triggers communications with multiple third-party services, including advertisement networks, analytics platforms, content delivery networks (CDNs), tracking services, and external APIs. As a result, browsing traffic is not limited to the primary visited domain but instead generates complex communication patterns involving many auxiliary domains. This behavior directly motivates the feature extraction strategy adopted in this work, where DNS queries, server IP sequences, and SNI values are analyzed together in order to capture the broader structure of the browsing session rather than focusing solely on the main domain.

Furthermore, *A World Wide View of Browsing the World Wide Web* [3] demonstrated that browsing behavior varies significantly depending on geographic regions, devices, and browsing contexts, reinforcing the idea that web traffic analysis must account for substantial variability and heterogeneity. These findings support the use of scalable and automated data collection pipelines capable of generating large and diverse traffic datasets under controlled conditions while still preserving realistic browsing characteristics.

Synthetic browsing is typically implemented through browser automation technologies capable of emulating user interaction with websites. Tools such as Selenium, Puppeteer, Playwright, and OpenWPM allow researchers to automatically launch browser sessions, navigate webpages, click buttons, complete forms, and reproduce browsing sessions at scale. Similarly, industrial automation platforms also provide advanced browser automation capabilities. For example, the Microsoft documentation *Automa-*

te Webpages - Power Automate [4] describes how Power Automate can automatically launch browsers, interact with webpage elements, extract structured data, navigate across multiple pages, and communicate with web services and APIs through automated workflows . These technologies demonstrate that automated browsing infrastructures have become mature and scalable solutions for generating reproducible web interaction datasets.

Although synthetic browsing provides scalability and experimental control, modern web privacy research also relies on real user datasets to understand browsing behavior under realistic conditions. In the study *Browsing behavior exposes identities on the Web* [5] analyzed browsing traces collected from 2,148 real users in Germany, containing more than 9.1 million website visits across approximately 50,000 unique domains. The dataset included timestamps, visited domains, browsing frequency, and session information while preserving anonymity through GDPR-compliant collection procedures. Using this dataset, the authors demonstrated that habitual browsing behavior creates highly distinctive behavioral fingerprints capable of identifying users with only a small number of visited domains.

Together, these studies illustrate how modern web privacy research increasingly combines controlled synthetic browsing infrastructures with large-scale real-world behavioral datasets in order to better understand encrypted traffic generation, browsing fingerprintability, and the privacy risks associated with modern web activity. These real datasets have revealed that browsing behavior itself can become a privacy vulnerability.

2.2. Web Browsing and Application Identification

Traditionally, identifying which websites or applications users accessed relied on Deep Packet Inspection (DPI), where network devices inspected packet payloads, HTTP headers, URLs, DNS queries, and protocol signatures to directly determine the generated service or application. This approach was highly effective when most Internet traffic was transmitted in plaintext. However, the widespread adoption of encryption technologies such as HTTPS, TLS 1.3, QUIC, DNS over HTTPS (DoH), and Virtual Private Networks (VPNs) has significantly reduced packet visibility, making payload-based identification increasingly ineffective.

To overcome these limitations, modern traffic analysis techniques focus on metadata-based identification rather than direct payload inspection. Even when traffic is encrypted, network flows still expose observable characteristics such as packet size, packet direction, flow duration, inter-arrival times, and byte distribution patterns. These statistical properties can reveal behavioral fingerprints associated with specific applications or traffic categories. According to *VPN Traffic Analysis: A Survey on Detection and Application Identification* [6], early encrypted traffic classification approaches relied mainly on handcrafted statistical features combined with traditional machine learning algorithms such as Random Forest (RF), Decision Trees (DT), Support Vector Machines (SVM), and k-Nearest Neighbors (KNN). The-

se methods were commonly used to classify traffic into categories such as web browsing, streaming, messaging, file transfer, or Voice over IP (VoIP) traffic.

More advanced approaches later incorporated deep learning techniques capable of automatically extracting relevant patterns directly from network traffic. The authors proposed one of the earliest end-to-end encrypted traffic classification methods based on one-dimensional Convolutional Neural Networks (1D-CNNs), where raw packet sequences are processed directly without requiring manual feature engineering [7]. Their work demonstrated that CNN-based architectures can successfully learn discriminative patterns from encrypted traffic flows and outperform traditional machine learning methods in terms of accuracy and scalability. This represented a significant shift from handcrafted statistical analysis toward automated feature learning.

Following this direction, recent studies surveyed by Razooqi and Pekar introduced increasingly sophisticated deep learning architectures for encrypted application identification [6]. These include Convolutional Neural Networks (CNNs), Long Short-Term Memory (LSTM) networks, attention-based models, autoencoders, Graph Neural Networks (GNNs), and transformer-based architectures such as DistilBERT and T5. Long Short-Term Memory (LSTM) networks are a specialized type of recurrent neural network designed to capture long-term temporal dependencies in sequential data, making them particularly suitable for analyzing packet sequences and traffic flows over time. These models can learn complex temporal and structural relationships directly from encrypted traffic without relying on manually designed features.

Several approaches also transform traffic into alternative representations to improve classification performance. Some methods convert packet sequences into images processed by CNNs, while others represent traffic as time series or graph structures to capture temporal and relational dependencies between packets. More recently, Graph Neural Networks and transformer-based models have demonstrated strong performance in identifying encrypted applications by modeling complex interactions within traffic flows and learning contextual packet relationships more effectively.

Despite the significant progress achieved by modern machine learning and deep learning techniques, encrypted application identification remains a challenging problem. Continuous changes in encryption protocols, VPN tunneling mechanisms, traffic obfuscation strategies, and increasingly heterogeneous user behavior reduce the effectiveness of static classification models. As discussed by Razooqi and Pekar, current research therefore focuses on developing more robust, adaptive, and scalable models capable of maintaining high classification accuracy under dynamic and heavily encrypted network environments.

2.3. Cookies and Web Session Management

The Hypertext Transfer Protocol (HTTP) is inherently stateless, meaning that each request between a browser and a web server is processed independently without retaining information about previous interactions. While this simplifies communication, modern web applications require mechanisms to preserve user state across multiple pages for functionalities such as authentication, personalization, and session management. To address this limitation, websites rely on client-side storage technologies that allow information to persist during navigation.

Cookies are the most common mechanism used to maintain state on the web. A cookie is a small piece of data created by a web server and stored in the user's browser after the server sends a `Set-Cookie` header in an HTTP response. The browser stores this information and automatically includes it in future requests to the same domain through the `Cookie` header, allowing websites to recognize returning users and maintain session continuity. Cookies typically store session identifiers, authentication tokens, user preferences, or tracking information. *How We Use Cookies* [8] describes cookies as small pieces of data stored on the user's device to recognize repeat visitors and improve service functionality, analytics, and personalization. Similarly, *Cookies Used on Survey Pages* [9] explains that cookies are necessary to securely operate survey services, maintain stability, and support authentication and session continuity.

Different types of cookies exist depending on their purpose. Session cookies are temporary and are deleted when the browser closes, typically used to maintain login sessions or temporary shopping cart information. Persistent cookies remain stored for a predefined period and are commonly used to remember user preferences or support long-term tracking. Cookies can also be classified as first-party cookies, created by the website being visited, or third-party cookies, created by external services such as advertising networks, analytics providers, or embedded platforms. The *How We Use Cookies* [8] documentation provides practical examples of these categories, including authentication cookies, analytics cookies from Google Analytics, Microsoft Clarity tracking cookies, advertising-related cookies from Facebook and Microsoft, and security cookies associated with Cloudflare services. *Cookies Used in Survey Pages* [9] similarly distinguish between required cookies, functional cookies, and advertising cookies used for analytics, marketing effectiveness, and monitoring of user behavior across websites.

Although cookies remain widely used, modern browsers also provide additional storage mechanisms. Local Storage allows websites to store larger amounts of persistent data directly in the browser, while Session Storage only preserves information during the current browsing session. More complex applications may use IndexedDB to maintain structured local databases, and browser caches store resources such as images, scripts, and HTML files to improve performance. Unlike traditional cookies, some of these mechanisms do not automatically send data in every HTTP request, but they still allow websites to maintain user state. *Journal of Open Innovation: Technology, Market, and Complexity* [10] explain that modern web tracking increasingly combines traditional cookies with alternative browser

storage technologies and fingerprinting techniques to improve persistence and user identification capabilities in modern web environments.

In modern stateless web architectures, JSON Web Tokens (JWT) have emerged as a dominant compact and self-contained format for transmitting authenticated information securely between clients and servers, standardizing decentralized token-based authentication. As *What is a JWT (JSON Web Token): How it Works, Vulnerabilities, and Best Practices* [11] explains, a JSON Web Token is an open standard (RFC 7519) that defines a compact and self-contained way for securely transmitting information between parties as a JSON object. This information can be verified and trusted because it is digitally signed. They are structured as a single string divided into three distinct Base64URL-encoded components separated by dots: the header, the payload, and the signature. According to *Analysis of Compromised User Authentication Threats for JSON Web Tokens* [12], a JWT encapsulates all necessary user data or authorization statements (*claims*) within itself. This architecture enables stateless verification, meaning that the backend application can validate the authenticity of the token solely by checking its cryptographic signature without making repetitive database queries to look up an active user session (mentioned in [11, 12]). For secure operation, the signature can be generated using symmetric algorithms like HS256, where a single shared secret key is utilized for both signing and verification, or asymmetric options like RS256, which divide duties between a public key and a private key.

Despite their architectural efficiency, the implementation of JWTs introduces a significant attack surface if security protocols are neglected. Both [11, 12] cite that a fundamental risk inherent to the format is that the payload containing the user claims is only encoded rather than encrypted; therefore, any party in possession of the token can easily decode and read its text contents, prohibiting the storage of sensitive data like passwords inside the token. Furthermore, cryptographic implementations are frequently vulnerable to authentication compromises, which rank among the top security risks in modern Application Programming Interfaces (APIs) according to the Open Web Application Security Project (OWASP). Exploitable implementation flaws include the accepted use of the `alg: none` header variant that allows attackers to strip signatures completely, or the deployment of short, low-entropy keys that are susceptible to offline brute-force cracking using standard auditing software. Additionally, because a signed token remains valid until its pre-defined expiration timestamp (`exp`), capturing a long-lived token via client-side storage vulnerabilities or phishing campaigns grants an adversary persistent unauthorized account access, necessitating brief lifespan windows for access tokens and robust revocation strategies.

According to *Microsoft Identity Platform and OAuth 2.0 Authorization Code Flow* [13], the procurement and distribution of these cryptographic tokens across secure enterprise applications are governed by formal architectural frameworks, most notably the OAuth 2.0 authorization protocol. While JWT defines the structure and format of the token itself, OAuth 2.0 establishes the specific systemic flows required to safely issue and exchange them [11]. Within this framework, the OAuth 2.0 authorization

code flow acts as the primary grant type designed to allow client applications (ranging from single-page web applications (SPAs) to native desktop and mobile platforms) to obtain authorized access to protected backend APIs on behalf of a user. This protocol sequence separates the user authentication process from the client application by utilizing a trusted intermediary authorization server, which securely redirects an authorization code back to the client application via a compatible user-agent or web browser [13]. To protect this redirection sequence against interception vectors in public client environments, the flow is structurally paired with the Proof Key for Code Exchange (PKCE) extension alongside OpenID Connect (OIDC) identity layers, guaranteeing that only the specific client instance that initiated the request can successfully exchange the temporary code for a high-privilege access token [13].

These storage mechanisms and authorization frameworks are directly connected to web navigation behavior. As users move between pages or websites, stored identifiers allow platforms to recognize users, maintain active sessions, record visited pages, and personalize future interactions. In many cases, analytics and advertising services use these identifiers to reconstruct browsing histories and build behavioral profiles across multiple websites. *How We Use Cookies* [8] explicitly mentions that cookies are used to collect information about visited pages, visitor origins, clicks, session duration, device information, and interactions with advertisements. Likewise, *Cookies Used in Survey Pages* [9] states that advertising networks may follow users' online activities over time by collecting navigational information through cookies and related technologies. According to *Journal of Open Innovation: Technology, Market, and Complexity* [10], these tracking capabilities create significant privacy concerns because users are often unaware of the amount of behavioral information continuously collected during normal web browsing activity.

3

Methodology

This chapter presents the methodology followed to design, implement, and evaluate the proposed solution. It describes the complete workflow adopted throughout the project, from the collection of web browsing data to the analysis and modeling stages. The chapter is structured into several sections that detail how browsing data is automatically obtained, how network traffic is processed and transformed into structured datasets, and how relevant features are extracted and represented. Finally, the machine learning approach used to analyze the data is described, providing the basis for the experimental evaluation presented in later chapters.

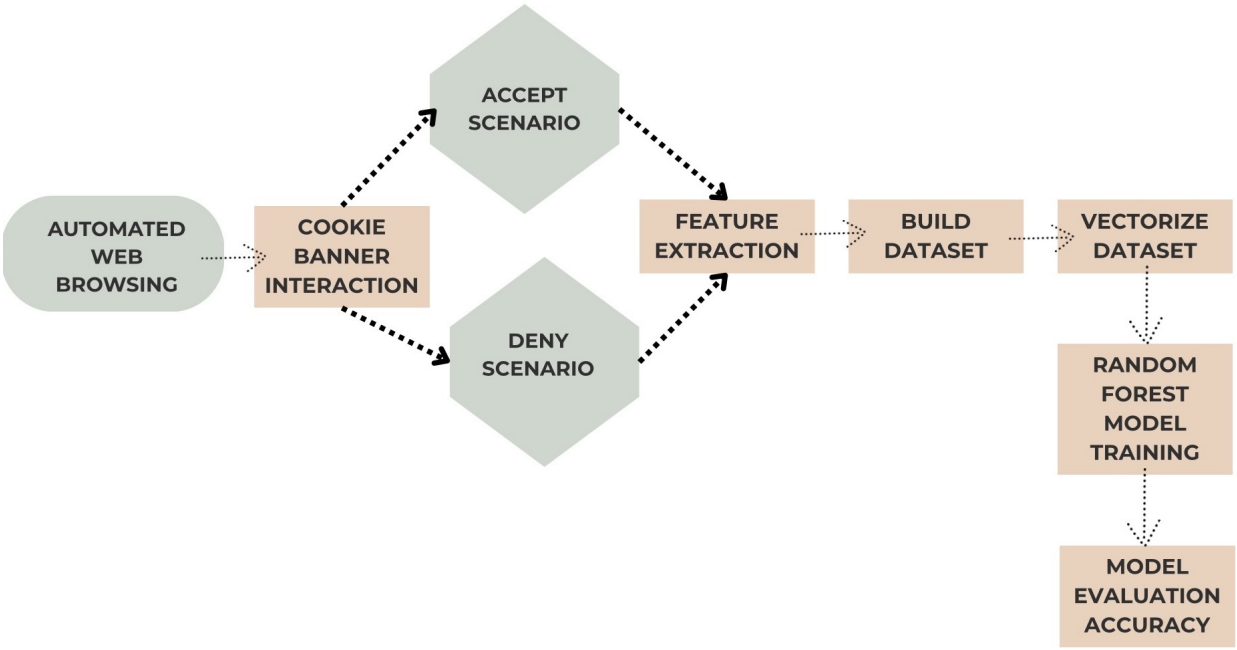


Figure 3.1: *Overview of the methodology pipeline from data collection to model evaluation.*

The following example illustrates the complete transformation pipeline applied to a browsing session throughout the project, from the original network traffic captured in a PCAP file to the final numerical representation used by the Random Forest classifier.

First, the raw network traffic is processed during the feature extraction stage. Multiple traffic-related features are extracted from the PCAP capture, including packet statistics, DNS queries, server IP ad-

dresses, and TLS SNI values. For example, a browsing session associated with YouTube could generate the following sequences:

```
Domain:
youtube.com

Server IP sequence:
142.250.184.78 142.250.184.110 172.217.168.14 ...

DNS query sequence:
youtube.com googlevideo.com ytimg.com
youtubei.googleapis.com doubleclick.net ...

SNI sequence:
youtube.com googlevideo.com ytimg.com ...
```

At this stage, the data is still represented as plain text sequences. Since machine learning models cannot directly process textual data, the next step consists of converting these sequences into numerical representations using the HashingVectorizer.

The HashingVectorizer transforms each token into a fixed numerical index through a hashing function. Conceptually, the previous sequences could be transformed into something similar to the following:

```
Hashed token representation:

youtube.com                → 12
googlevideo.com            → 1239
ytimg.com                  → 2131
youtubei.googleapis.com    → 323
doubleclick.net            → 10
```

After hashing all tokens, the vectorizer generates a sparse numerical vector representing the frequency of the hashed features within the browsing session. A simplified example would be:

```
Sparse hashed vector:

[0, 1, 0, 2, 0, 0, 1, 0, 0, 3, ...]
```

This vector contains the frequency distribution of the hashed tokens inside a high-dimensional feature space. In the implemented system, separate vectorizations are generated for server IP sequences,

DNS query sequences, and SNI sequences, which are later concatenated together.

Before adopting the hashing-based representation, embedding-based approaches were also evaluated experimentally. In that case, each domain token was converted into a dense numerical vector intended to capture semantic similarities between elements. A simplified example of these embeddings would be:

```
Embedding representation:

youtube.com              → [9.1, -1.2, 0.4, ...]
googlevideo.com          → [8.1,  2.0, -3.0, ...]
ytimg.com                → [4.2, -0.7, 1.8, ...]
```

However, since domains, IP addresses, and network identifiers do not contain semantic relationships comparable to natural language, the embedding-based approach produced worse results than the frequency-based hashing representation.

Finally, all extracted information is merged into a single feature vector together with the numerical traffic statistics extracted from the PCAP capture. The resulting representation corresponds to the final input fed into the Random Forest classifier. A simplified example of this final vector would be:

```
Final Random Forest input vector:

[
  total_packets = 542,
  total_bytes = 182394,
  avg_packet_size = 336.5,
  tcp_packets = 498,
  dns_packets = 12,
  duration_seconds = 18.2,
  ip_12 = 1,
  ip_323 = 2,
  dns_1239 = 3,
  sni_2131 = 1,
  ...
]
```

This final numerical representation encodes both the statistical properties of the traffic and the structural communication patterns associated with the browsing session. The Random Forest model then uses these vectors to learn discriminative patterns capable of identifying the visited domain.

3.1. Web browsing data

This section describes the process followed to collect and prepare web browsing data for analysis. First, the mechanism used to automatically accept or deny cookies during navigation is presented, detailing how user interactions are simulated in a controlled browsing environment. Subsequently, the procedure for capturing and processing network traffic is explained, outlining how raw packet data is transformed into structured datasets suitable for further analysis.

3.1.1. Cookie Consent Interaction Methodology

In this work, cookie consent is handled automatically in order to collect network traffic under two controlled conditions: browsing sessions in which cookies are accepted and browsing sessions in which cookies are rejected. The main objective of this process is to generate comparable network traffic captures that allow analyzing whether the visited domain can be inferred from traffic characteristics depending on the consent choice.

The browsing sessions are performed over a predefined set of websites corresponding to the 1,000 most visited domains. This selection ensures that the analysis focuses on popular and representative websites, while also providing a diverse range of content, infrastructures, and third-party services. Each domain is visited independently in multiple automated browsing sessions, and a single browsing session produces one network traffic capture.

To automate the browsing process, Selenium is used as the browser automation framework. Selenium allows programmatic control of a real web browser, enabling the simulation of user actions such as loading pages, waiting for elements to appear, and clicking interface components. In this project, Selenium is executed in headless mode, meaning that the browser operates without a graphical user interface while maintaining full browsing functionality. Communication with the browser is managed through WebDriver, specifically using ChromeDriver, which is automatically configured through WebDriver Manager to ensure compatibility between the browser and the automation driver.

Once a webpage is loaded, the system checks whether a cookie consent banner is displayed. Due to the lack of standardization in cookie banner implementations, a heuristic approach based on keyword matching is adopted instead of site-specific rules. Two separate sets of consent-related keywords are defined and stored externally. The first set corresponds to cookie acceptance actions (e.g., *accept, agree, allow*), while the second set corresponds to cookie rejection actions (e.g., *deny, reject, reject all, decline*). Depending on the target scenario of the browsing session, the appropriate keyword set is loaded at runtime.

The system then inspects the Document Object Model (DOM) of the webpage to locate clickable elements, focusing on buttons. For each candidate element, the visible text is extracted and compared

against the selected keyword list using case-insensitive matching. When a matching element is found, Selenium ActionChains are used to simulate a realistic user interaction by moving to the element and triggering a click. This approach improves robustness when dealing with dynamically rendered content or elements that require user-like interaction. Controlled delays are also introduced to ensure that cookie banners, which are often loaded asynchronously, are fully rendered before detection and interaction.

Not all browsing attempts result in a valid session. Domains are explicitly excluded from the data collection process when the webpage fails to load correctly, when no cookie banner is detected, or when none of the banner elements match the predefined acceptance or rejection keywords. This filtering step is essential to guarantee that each recorded session corresponds to a successful and explicit consent decision. Only sessions in which the consent action is clearly executed are retained for further analysis.

For each valid browsing session, network traffic is captured in a *.pcap* file covering the entire interaction, including page loading and the cookie consent decision. As a result, two independent collections of traffic captures are obtained: one corresponding to sessions in which cookies are accepted and another corresponding to sessions in which cookies are rejected. These captures represent synthetic but realistic browsing traffic and form the basis of the subsequent data processing pipeline.

The collected *.pcap* files are later processed to build structured datasets and are used to train and evaluate machine learning models aimed at predicting the visited domain. This evaluation is performed independently for the acceptance and rejection scenarios, allowing a direct comparison of the prediction accuracy under different cookie consent conditions.

3.1.2. Network Packet Processing

The dataset construction process starts from the network traffic generated during automated web browsing experiments. Each experiment consists of visiting a website and interacting with its cookie consent banner, while all network communications are recorded. Traffic is captured in .pcap files, where each file corresponds to one complete browsing session for a single website, including the initial page load and the moment at which the cookie consent decision is performed.

The experimental protocol is executed sequentially for the two consent conditions. First, the experiment is run under the accept scenario, in which the cookie banner is accepted for all websites. Once this execution is completed, the same experiment is repeated under the deny scenario, where the cookie banner is rejected. Each scenario stores its .pcap files in a separate folder, ensuring that traffic captures from different consent conditions are never mixed.

After data collection, each execution folder is processed independently to generate a structured dataset. The processing script reads all *.pcap* files contained in a given folder, and produces one CSV dataset per folder. As a result, two independent datasets are obtained: one associated with the accept condition and one associated with the deny condition. In all cases, each row of the dataset represents

one browsing session.

For every *.pcap* file, the script parses the captured packets using Scapy and computes a fixed set of features that summarize the session at multiple levels. First, general traffic statistics are extracted, including the total number of packets and total number of bytes exchanged during the session, as well as descriptive statistics of packet sizes (mean, standard deviation, minimum, and maximum). These features provide a compact representation of the overall traffic volume and variability generated by a browsing session.

In addition, protocol-level information is derived by counting the number of packets containing TCP, UDP, and DNS layers. This allows distinguishing between transport-layer communication and name resolution activity. The temporal extent of the session is computed as the difference between the timestamps of the first and last packet in the capture, yielding the total duration of the browsing session.

Flow-level behavior is captured through the number of TCP flows observed in each session. A flow is defined as a unique four-tuple composed of source IP address, destination IP address, source port, and destination port. The total number of distinct TCP flows provides an approximation of how many parallel connections are established while interacting with the website. Directional traffic characteristics are also estimated. When the local client IP address is available, the script computes the number of bytes sent and received by the client and derives an upload–download ratio. When the local IP is not specified, all traffic volume is conservatively assigned to the outgoing direction. This feature captures the balance between client requests and server responses during the session.

Beyond aggregate numerical features, the processing step also extracts sequential information that preserves the order of relevant network interactions. Three ordered sequences are generated for each session. First, the sequence of destination IP addresses contacted during the session is recorded, reflecting the servers involved in the communication. Second, the sequence of DNS query names is extracted from DNS request packets, capturing the domains resolved while loading the website. Third, when TLS traffic is present and supported by the parsing library, the sequence of Server Name Indication (SNI) values observed in TLS ClientHello messages is collected. These sequences are stored as text fields, maintaining their original order of appearance in the traffic capture.

Finally, all extracted features (both numerical summaries and ordered sequences) are combined into a single record corresponding to the processed *.pcap* file. Each record is appended to the CSV dataset associated with the execution folder. The resulting datasets provide a structured and machine-learning-ready representation of raw network traffic, while preserving both statistical properties and behavioral patterns of web communication under different cookie consent conditions.

3.2. Data Analysis and Feature Extraction

This section presents the data analysis pipeline and the feature extraction techniques applied to the network traffic datasets generated in the experiments. It first introduces the transformation of sequential network features into numerical representations through embedding techniques, which enable their use in machine learning models. Different embedding strategies are discussed, highlighting their trade-offs in terms of interpretability, scalability, and representational power. Subsequently, the machine learning approach adopted in this work is described, including the selected model, the training and evaluation procedure, and the criteria used to assess classification performance.

3.2.1. Embeddings

In order to make use of sequential information extracted from network traffic in machine learning models, the raw sequences must be transformed into fixed-size numerical representations. The datasets generated from packet captures include several sequential features. Specifically, the ordered sequences of contacted server IP addresses, DNS query names, and TLS Server Name Indication (SNI) values. These features are inherently variable in length and textual in nature, which prevents their direct use in traditional machine learning algorithms. For this reason, an embedding step is required.

In this work, hashing-based vectorization is adopted as the embedding strategy for sequential features. This choice is motivated by the characteristics of network traffic data, where the number of possible tokens (e.g., IP addresses or domain names) can be extremely large and continuously evolving. Approaches that rely on building and maintaining an explicit vocabulary become impractical in this context, both in terms of memory usage and preprocessing complexity.

The hashing vectorization technique maps each token in a sequence to a fixed-dimensional numerical space using a hash function. Rather than assigning each unique token a specific index in a learned vocabulary, the hash function deterministically converts tokens into indices within a predefined vector size. As a result, the dimensionality of the output vectors remains constant, independently of the number of distinct tokens observed in the dataset. This property makes the method highly scalable and well suited for network traffic analysis.

For each dataset, previously generated vectorized files are removed in order to avoid inconsistencies and ensure that embeddings are always computed from the original, non-vectorized data. Then , the CSV datasets are identified within the selected experimental folder (either corresponding to the accept or deny scenario) and processes them independently.

Three separate hashing vectorizers are created, one for each sequential feature: server IP sequences, DNS query sequences, and SNI sequences. In all cases, the same configuration is applied. The sequences are tokenized by splitting on whitespace, preserving the original order in which elements

appeared in the network capture. Each token is then mapped into a vector of fixed dimensionality, defined as 2^{12} features. This dimensionality represents a compromise between representational capacity and computational efficiency, as larger vector sizes increase both memory consumption and processing time.

The vectorization process converts each sequence into a numerical vector where each component represents the frequency of hashed tokens mapped to that position. The sign alternation commonly used in hashing vectorization is disabled, ensuring that all feature values are non-negative and directly interpretable as aggregated counts. No normalization is applied at this stage, since the objective is to preserve raw occurrence patterns rather than relative frequencies.

Once the three sequential features have been vectorized, the original textual columns are removed from the dataset. The resulting numerical vectors are then concatenated with the remaining statistical features extracted from the traffic captures, such as packet counts, protocol distributions, flow-based metrics, and temporal information. This produces a unified feature space that combines both low-level traffic statistics and high-level behavioral sequences.

Finally, the fully vectorized dataset is stored as a new CSV file, clearly distinguished from the original dataset by an explicit suffix in its filename. The output datasets are directly suitable for machine learning training and evaluation, as all features are now represented in numerical form with consistent dimensionality across all browsing sessions and experimental conditions.

3.2.2. Machine Learning Model

To evaluate whether network traffic features can be used to discriminate between different websites, a supervised machine learning approach is adopted. In this work, a Random Forest classifier is selected as the learning model due to its robustness, ability to handle high-dimensional data, and suitability for combining heterogeneous feature types.

The model operates on the vectorized datasets obtained after the embedding step. Each instance in these datasets corresponds to a single browsing session and is represented by a combination of numerical traffic statistics and hashed embeddings derived from sequential features (server IPs, DNS queries, and SNI values). Separate experiments are conducted for each consent condition (accept and deny), using the corresponding vectorized datasets.

Since multiple dataset files may originate from the same base execution (for example, due to data augmentation or repeated captures), the datasets are first grouped according to their original base name. This grouping is preserved when splitting the data into training and testing sets in order to prevent samples from the same execution appearing in both partitions. The groups are randomly shuffled using a fixed random seed to ensure reproducibility, and then split into training and testing subsets using an 80/20 ratio. A safeguard is applied to guarantee that the test set is never empty. All datasets belonging

to the training groups are merged into a single training dataframe, and the same procedure is applied to the test groups. Before training the model, a class reduction strategy is applied to address class imbalance and limit the complexity of the classification task. Specifically, only the top 20 most frequent domains in the training data are retained as individual classes (more classes can be defined depending on the execution complexity). All remaining domains are grouped into a single generic class labeled other. This transformation is applied consistently to both training and testing datasets and results in a controlled and comparable set of output classes.

After class adjustment, the feature matrix and target labels are separated. All columns except the domain label are used as input features, while the domain column represents the classification target. The final feature space includes a large number of dimensions due to the hashed embeddings; however, Random Forests are well suited to this scenario, as they perform implicit feature selection through tree-based splitting.

Random Forest is an ensemble learning algorithm based on the construction of multiple decision trees. Instead of relying on a single tree, the model generates a collection of trees during training and combines their predictions through majority voting. Each tree is trained using a random subset of the training samples and a random subset of the available features, introducing diversity among the individual learners. This strategy reduces overfitting and improves the generalization capability of the model compared to a single decision tree.

In the context of this work, Random Forest is particularly suitable because the generated feature space is high-dimensional and heterogeneous, combining statistical traffic metrics with hashed representations of DNS, SNI, and server IP sequences. Tree-based models can naturally handle sparse and non-linear feature interactions without requiring feature normalization or complex preprocessing. Furthermore, Random Forest performs implicit feature selection during the splitting process, meaning that only the most informative features are used to construct the decision boundaries.

The model is configured using a fixed number of estimators (`n_estimators=50`), meaning that fifty independent decision trees are generated during training. Additionally, the maximum depth of each tree is limited (`max_depth=15`) in order to reduce overfitting and avoid excessively specialized trees. A fixed random seed (`random_state=42`) is also employed to ensure reproducibility of the experiments. Finally, parallel execution is enabled through the `n_jobs=-1` parameter, allowing the training process to use all available CPU cores and reducing computational time.

The Random Forest classifier is configured with a fixed number of trees and a bounded maximum tree depth in order to balance expressive power and overfitting risk. Training is performed using all available CPU cores to improve computational efficiency. Once trained, the model is applied to the test set to generate predictions for unseen browsing sessions.

Model performance is evaluated using accuracy, defined as the proportion of correctly classified instances in the test set. This metric provides a straightforward assessment of the model's ability to

learn discriminative patterns from network traffic data and to generalize these patterns across different browsing sessions. Accuracy results are reported separately for each experimental condition, enabling a direct comparison of classification performance under different cookie consent scenarios.

4

Evaluation and Results

4.1. Elephants and Mice

The first step of the evaluation is to analyze the intrinsic characteristics of the collected web traffic dataset before applying any machine learning techniques. In particular, this analysis focuses on two complementary aspects: the distribution and persistence of domains within the traffic traces, and the size and variability of the resulting vocabulary of tokens extracted from encrypted traffic.

4.1.1. Traffic Distribution and Domain Persistence

During the analysis of the captured network traffic, an uneven distribution of domains was observed, where a small subset of services appears consistently across most captures, while others are only present under specific browsing conditions. This behavior can be described using the concept of *"Elephants and Mice"*.

In this context, elephants refer to domains that generate large volumes of traffic and appear persistently across different sessions. These domains are typically associated with widely used services such as content delivery networks, analytics platforms, or global providers. In the collected data, examples of such domains include *google.com, gstatic.com, or cloudflare.com*, which appear in most captures independently of the visited website. Their presence is not linked to a specific browsing action, but rather to general web infrastructure, as they are automatically requested by the browser when loading external resources required by the webpage, even if the user has not directly accessed those domains.

In contrast, mice correspond to domains that appear sporadically and are directly associated with the specific website being visited. These domains are only present in particular captures and vary depending on the navigation scenario. For example, domains such as *elpais.com* or *bbc.com* are only observed when those specific websites are accessed, making them highly dependent on the browsing context.

This behavior is clearly reflected in the dataset used in this work. Each browsing session is captured as a *.pcap* file and transformed into structured features, including sequences such as DNS queries,

server IPs, and TLS SNI values. Within these sequences, elephant domains tend to appear repeatedly across most captures, while mice domains vary significantly from one session to another.

Figure 4.1 illustrates this behavior using a session-domain presence heatmap. Each row represents a browsing session, while each column corresponds to a domain observed in the dataset. Bright regions indicate the presence of a domain within a session. On the left side of the figure, a small subset of domains appears consistently across nearly all sessions, forming dense vertical patterns. These correspond to elephant domains, which are associated with persistent web infrastructure services shared across websites. In contrast, the right side of the heatmap contains sparse and isolated activations, representing mice domains that only appear under specific browsing conditions. This visualization highlights the strong imbalance between persistent infrastructure-related traffic and context-dependent domains.

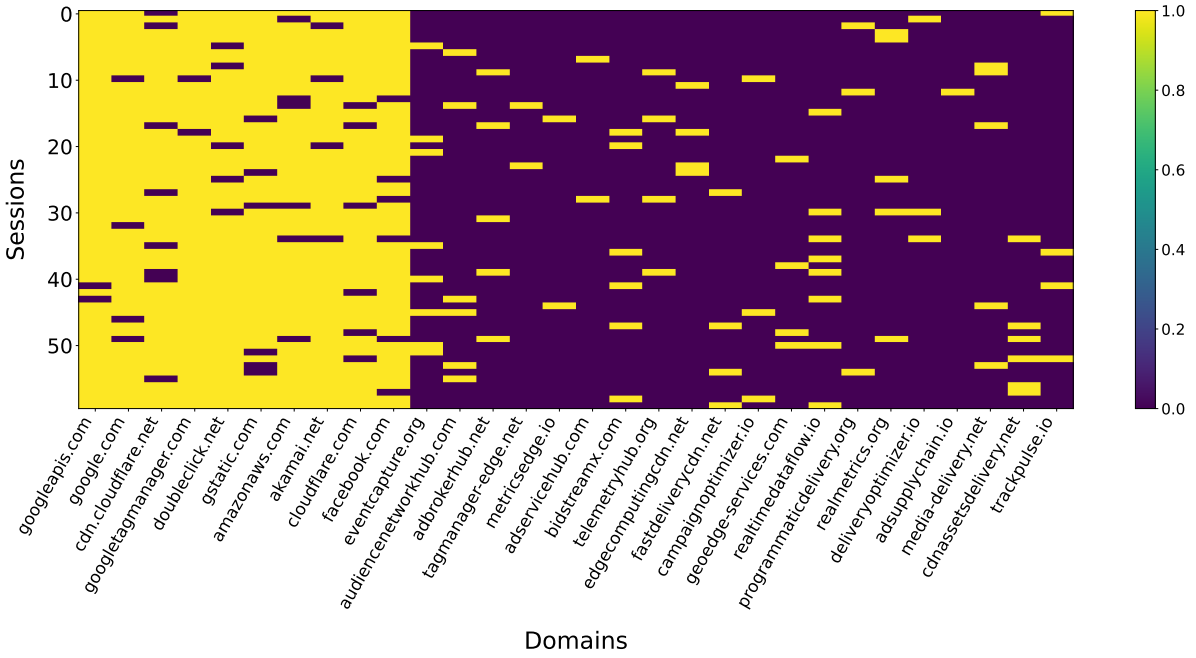


Figure 4.1: *Session-domain presence heatmap showing the difference between persistent elephant domains and sparse mice domains across browsing sessions.*

The same phenomenon can also be observed in the domain frequency distribution shown in Figure 4.2. The figure follows a clear long-tail behavior, where only a very small number of domains exhibit extremely high frequencies, while the vast majority appear only sporadically. The first domains in the ranking correspond to elephants and dominate a large portion of the observed traffic, whereas the long tail is composed of mice domains with very low occurrence frequencies. This type of distribution is commonly observed in large-scale network systems and confirms that most of the traffic is concentrated around a reduced set of shared services.

The results show that elephants dominate the traffic in terms of frequency, which can introduce bias in the cookie analysis. Due to their constant presence, these domains may have a strong influence

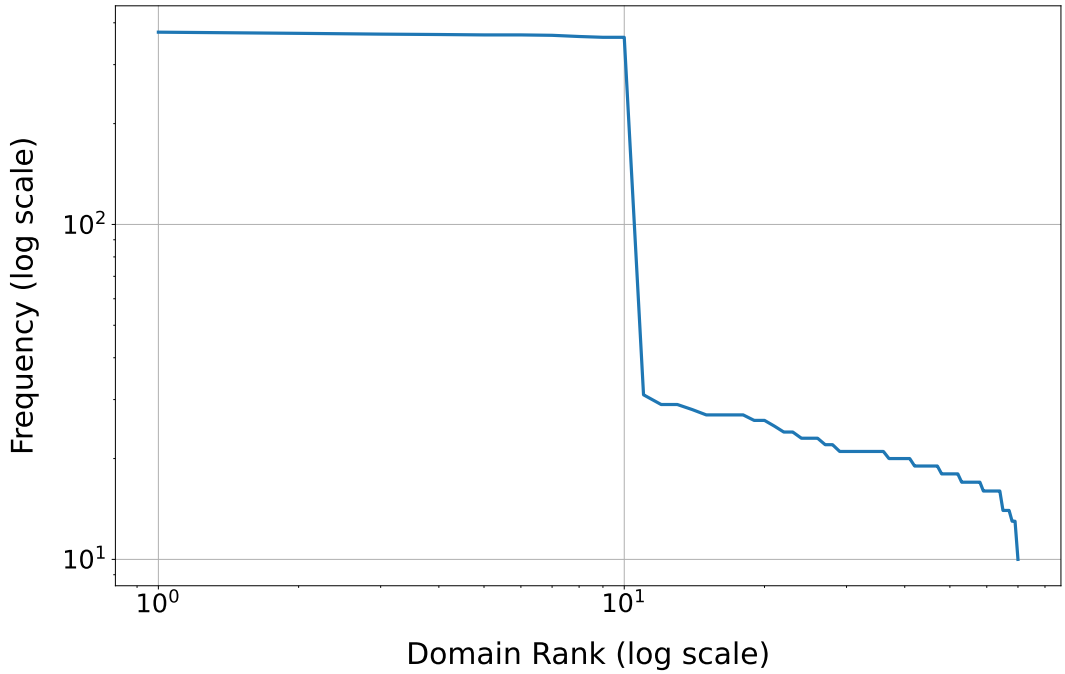


Figure 4.2: *Long-tail distribution of domain frequencies showing that a small subset of domains dominates the traffic while most domains appear infrequently.*

on statistical features and embedding representations, potentially leading models to focus on common infrastructure rather than session-specific behavior. This effect makes it more difficult to analyze the impact of accepting or rejecting cookies, since a large portion of the observed traffic remains similar across sessions regardless of the user's choice.

On the other hand, mice domains are the elements that define the unique characteristics of each browsing session. They reflect the actual content being accessed and provide the most relevant information for distinguishing between different navigation scenarios. Consequently, although elephants dominate the overall traffic volume, mice domains contain the most discriminative information for understanding browsing behavior and evaluating the real impact of cookie consent decisions.

4.1.2. The Vocabulary Problem

The application of embedding techniques to the extracted network data revealed several limitations directly related to the *Elephants and Mice* behavior described previously. In this work, embeddings are not applied to conventional textual data, but to sequences such as DNS queries, server IPs, and TLS SNI values, which represent the domains contacted during each browsing session. These sequences are characterized by the coexistence of highly frequent domains (elephants) and sporadic, session-specific domains (mice).

This distribution has a direct impact on how different embedding techniques perform. In particular, methods based on explicit vocabularies, such as Count Vectorization, are significantly affected. Since

elephant domains appear in almost every capture, they dominate the vocabulary and occupy a large portion of the feature space. As a result, the vocabulary grows considerably while many of its most frequent elements provide little discriminative value. This leads to inefficient representations, where common infrastructure domains introduce noise and reduce the ability to distinguish between different browsing sessions.

Figure 4.3 illustrates the evolution of the feature space size when using vocabulary-based representations compared to hashing-based embeddings as the number of processed sessions increases. In the case of Count Vectorization, the feature space continuously grows as new domains are observed in the traffic captures. This effect is mainly caused by the large number of low-frequency mice domains, which introduce thousands of additional tokens into the vocabulary over time. As a result, the representation becomes increasingly sparse and difficult to scale, since each newly observed domain requires the creation of an additional feature dimension.

In contrast, Hashing Vectorization maintains a fixed-dimensional representation independently of the number of observed sessions or domains. In this work, a HashingVectorizer with a fixed number of features is used in order to avoid the uncontrolled growth of the vocabulary and reduce memory requirements. Instead of explicitly storing every unique token, domains are mapped into a fixed set of feature positions through a hash function. This allows the representation size to remain constant even when the number of unique domains becomes very large.

The figure shows that the number of unique observed domains rapidly exceeds the dimensionality used by the hashing representation. Although this implies the existence of hash collisions, where multiple domains may share the same feature position, the trade-off significantly improves scalability and computational efficiency. This behavior is especially important in web traffic analysis, where a small number of highly repetitive elephant domains coexist with a very large long tail of sparse mice domains. Under these conditions, explicit vocabularies become increasingly inefficient, while hashing-based representations provide a more scalable solution for large-scale encrypted traffic classification.

The dominance of elephant domains within the feature space can be observed more clearly in Figure 4.4. A small subset of domains associated with shared web infrastructure services exhibits frequencies significantly higher than the remaining domains. Domains such as *googleapis.com*, *cloudflare.com*, or *amazonaws.com* dominate the representation despite not containing highly discriminative information about user navigation behavior. This imbalance causes embedding representations to focus excessively on common infrastructure-related traffic rather than session-specific patterns.

Similarly, more advanced approaches such as transformer-based embeddings present limitations in this context. Although these models are capable of capturing contextual relationships, they are primarily designed for natural language data. In this case, the input consists of domain sequences rather than linguistic structures, which reduces the effectiveness of contextual modeling. Additionally, the presence of highly repetitive elephant domains limits the contribution of meaningful variation, while the

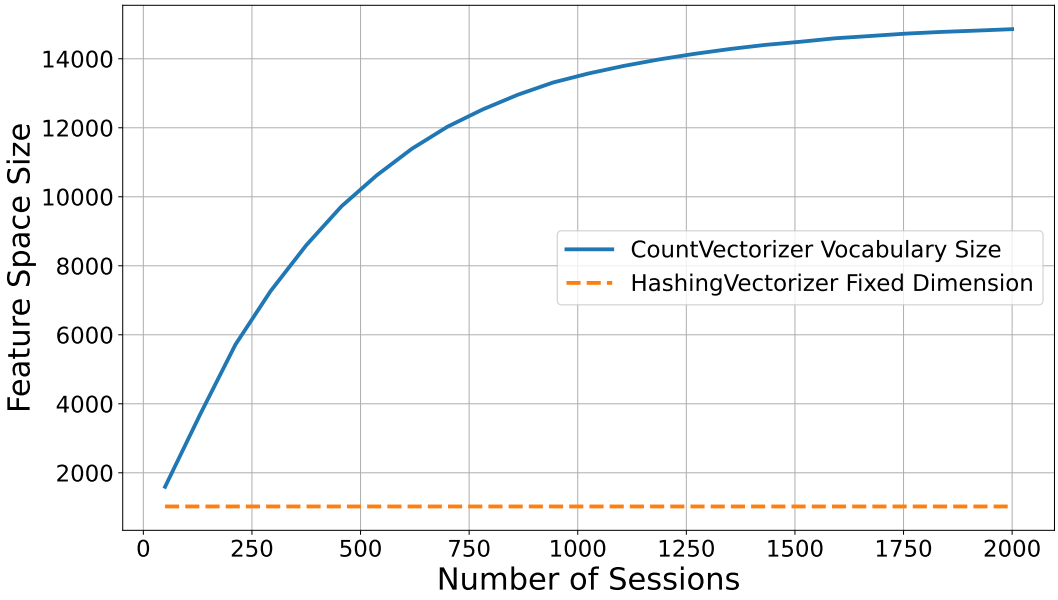


Figure 4.3: *Comparison between the vocabulary growth of Count Vectorization and the fixed-dimensional representation of Hashing Vectorization.*

computational cost of these models makes them less suitable for this type of data.

Overall, the results show that the presence of elephants imposes important constraints on embedding techniques. Not all methods are equally suitable for this type of data, and those relying on large vocabularies or contextual language modeling may be negatively affected. Consequently, selecting embedding strategies that are robust to high-frequency and repetitive patterns is essential for obtaining meaningful representations of network traffic.

4.2. Accept vs Deny vs No action

In this section, the results obtained from comparing two different cookie interaction scenarios are analyzed: cookie acceptance (Accept) and cookie rejection (Deny). The objective of this comparison is to evaluate whether the user's decision regarding cookie consent has a significant impact on the ability to identify the visited domain from the captured network traffic.

It is important to note that, although the experiments are grouped into Accept and Deny executions, some browsing sessions effectively correspond to a *No Action* scenario. This occurs due to limitations during the automated interaction process, such as websites without cookie banners, banners that failed to load correctly, or consent phrases that were not included in the predefined interaction dictionary. As a consequence, certain sessions in both datasets may contain traffic generated without any explicit interaction with the cookie banner.

The results indicate that there is no significant difference between the acceptance and rejection of

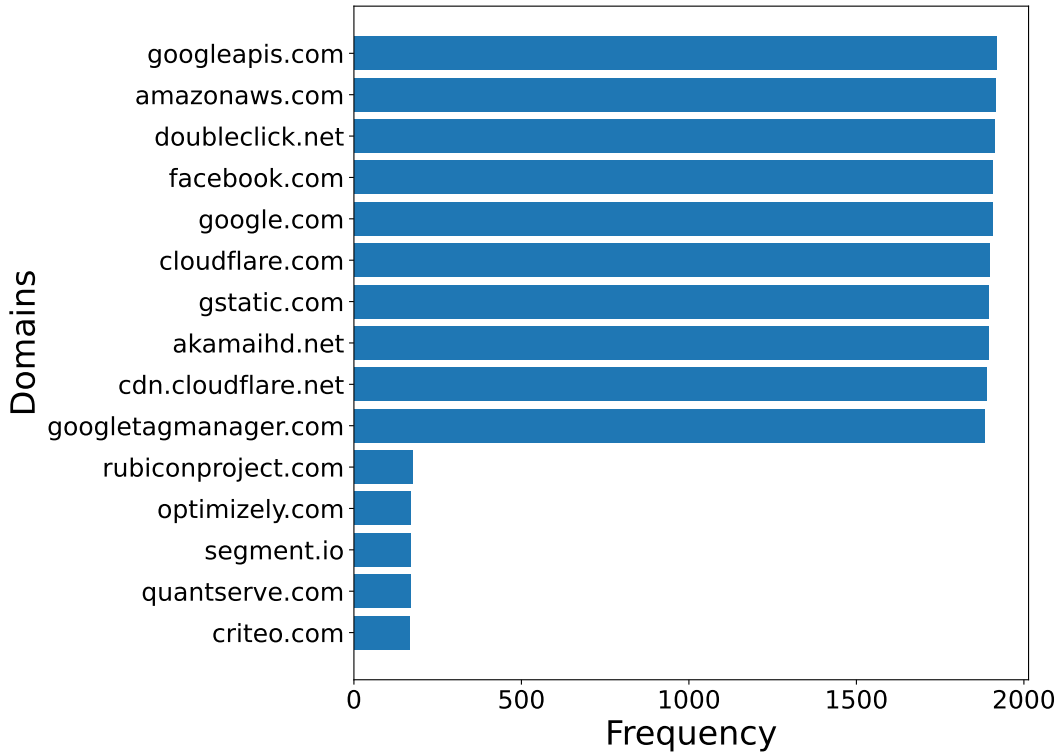


Figure 4.4: *Dominance of elephant domains within the feature space generated from browsing sessions.*

cookies. Across all experiments, in which the number of classes (Top-N domains) was varied to control the difficulty of the problem, the model achieved very similar accuracy values in both scenarios. This suggests that the information required to identify the domain does not depend directly on the presence or absence of cookies, but rather on other elements of the network traffic, such as DNS queries, SNI values, and communication patterns with external servers. Consequently, rejecting cookies does not lead to a meaningful reduction in the model's classification performance, indicating that cookies are not a determining factor in this context. It is also important to mention that a large number of features were extracted from the captured *.pcap* files, including DNS queries, SNI values, server IP sequences, and other communication patterns. Therefore, the model still has access to many different clues that can be used to infer the visited domain even when cookies are rejected.

Figure 4.5 summarizes the classification accuracy obtained under the Accept and Deny scenarios for different Top-N configurations. The results show that the accuracy progressively decreases as the number of classes increases, reflecting the growing complexity of the classification problem. In simpler scenarios such as Top-20, the model achieves high accuracy values close to 94 %, while in more realistic scenarios involving hundreds of domains, such as Top-400, the accuracy decreases significantly. Nevertheless, the Accept and Deny scenarios remain very close across all configurations, reinforcing the observation that cookie consent has only a limited influence on the identifiability of domains from encrypted network traffic.

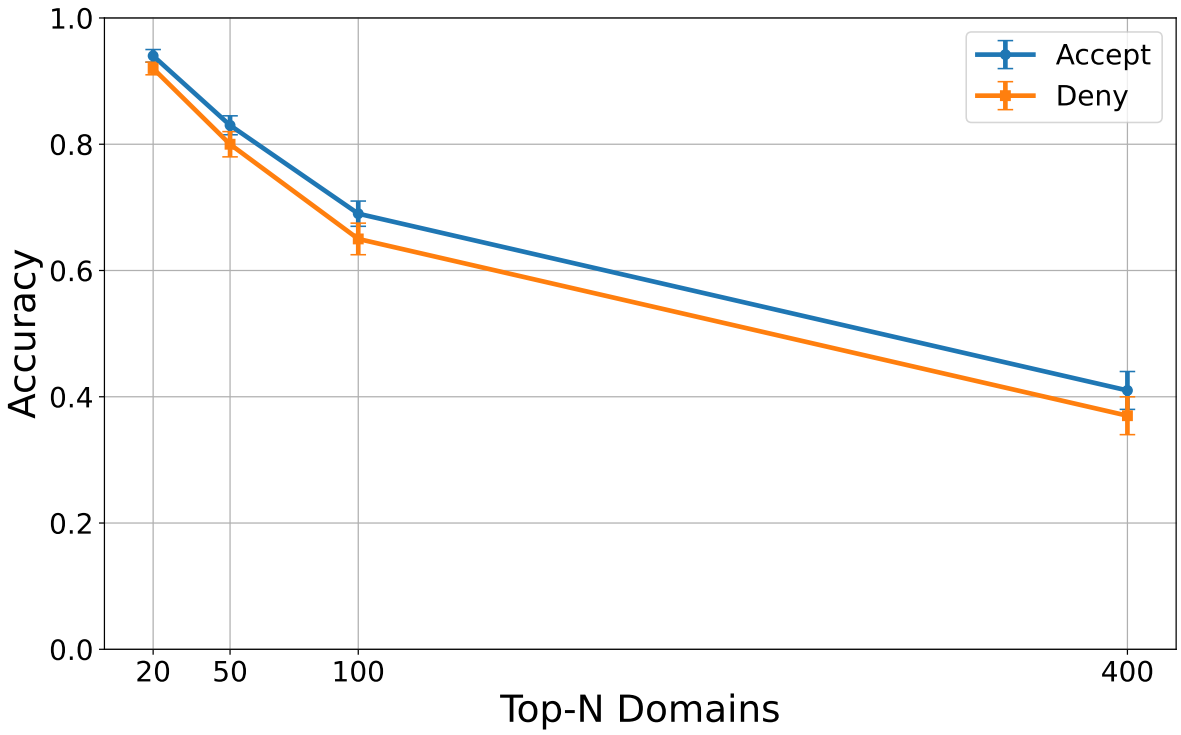


Figure 4.5: *Classification accuracy for the Accept and Deny scenarios under different Top-N domain configurations.*

Another relevant observation is that the results vary as the complexity of the problem increases. This complexity is controlled through the number of classes considered in the classification task (Top-N domains). When the number of classes is small, the model can identify clearer decision boundaries because each domain exhibits more distinguishable patterns and there are more samples available per class. However, as the number of classes increases, domains begin to exhibit more similar communication patterns, the number of samples per class decreases, and the probability of confusion between classes increases. Consequently, the Random Forest model encounters greater difficulty in partitioning the feature space effectively, resulting in lower accuracy values.

The relationship between the *Accept* and *Deny* scenarios can be observed more clearly in Figure 4.6, which shows the difference in accuracy between both configurations as the complexity of the problem increases. In simpler scenarios, such as Top-20, the difference is practically negligible. As the number of classes increases, a slight divergence becomes visible; however, this variation remains very small and statistically insignificant. This behavior reinforces the conclusion that the cookie consent state does not substantially affect the ability to identify visited domains from network traffic.

Although cookie consent is relevant from a privacy perspective, its impact on the identifiability of domains from network traffic is limited. Even in scenarios where cookies are rejected, the traffic still contains sufficient information to enable accurate classification. This finding reinforces the idea that web privacy cannot rely solely on cookie management, but must also consider other sources of information present in network traffic, such as metadata and communication patterns.

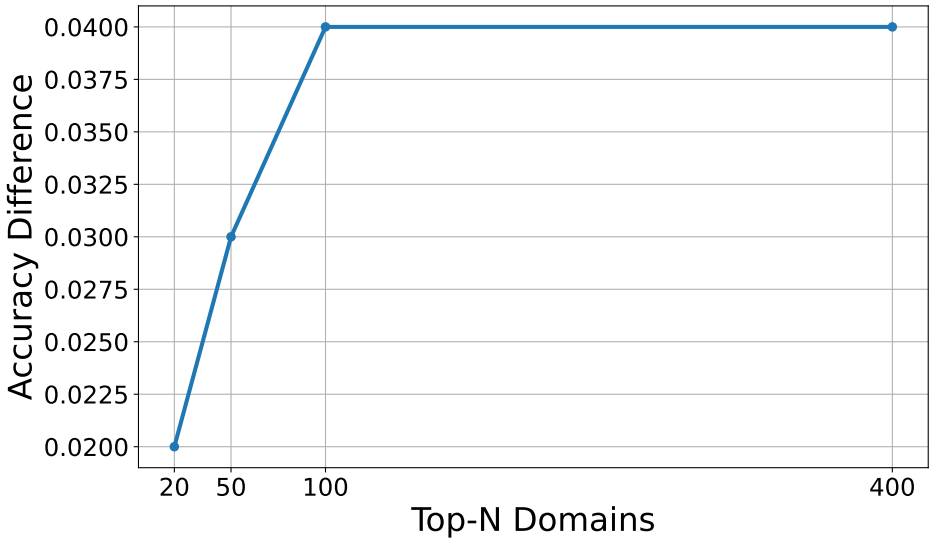


Figure 4.6: *Difference in classification accuracy between the Accept and Deny scenarios for different Top-N domain configurations.*

4.3. Impact of Classification Complexity

An additional key result of this work is the impact of problem complexity on the performance of the classification model. This complexity is controlled through the number of classes considered in the task, defined as the Top-N most frequent domains. By progressively increasing the number of classes, the classification problem becomes more challenging and better reflects realistic browsing scenarios.

The results show a clear degradation in performance as the number of classes increases. Figure 4.7 illustrates the evolution of the classification accuracy as the number of domains grows. In simpler configurations, such as Top-20, the model achieves high accuracy values close to 94 %. However, as the number of classes increases (Top-50, Top-100, and Top-400), the accuracy progressively decreases, reaching values close to 39 % in more complex settings. This behavior highlights the increasing difficulty of distinguishing between a larger number of domains based on network traffic features.

This effect can be explained by several factors. First, as the number of classes increases, the number of samples per class decreases, which limits the amount of information available for the model to learn representative patterns. Figure 4.8 shows the reduction in the distribution of samples per class for increasingly complex Top-N configurations. In scenarios with a limited number of classes, each domain is represented by a relatively large number of samples. However, as the number of classes increases, the available samples become significantly more sparse, reducing the model's ability to identify stable and discriminative traffic patterns.

Second, the similarity between classes also increases as many domains share common traffic characteristics, particularly due to the presence of shared infrastructure services. In this context, the previously identified *Elephants and Mice* behavior plays a significant role. Elephant domains, which appear

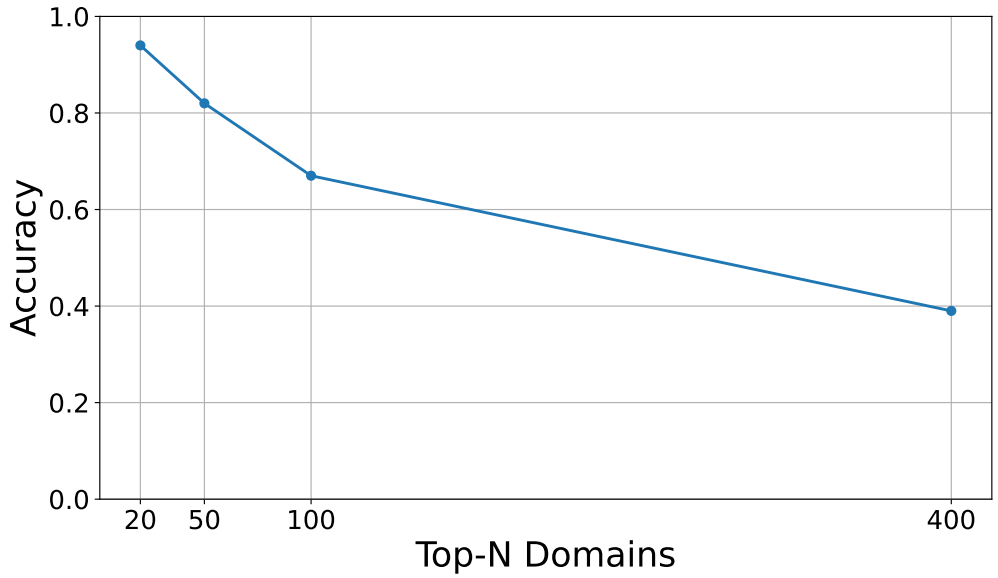


Figure 4.7: *Classification accuracy degradation as the number of classes increases.*

in most sessions, introduce common patterns across different classes and reduce the discriminative power of the features. At the same time, mice domains, which are specific to individual browsing sessions, become more numerous but are represented by fewer samples each. This combination makes it increasingly difficult for the model to identify clear decision boundaries as the number of classes grows.

These results demonstrate that the scalability of the classification task is limited by both the intrinsic characteristics of the network traffic and the increasing number of classes. As the problem becomes more complex, the model faces greater challenges in distinguishing between domains, leading to a natural decrease in accuracy.

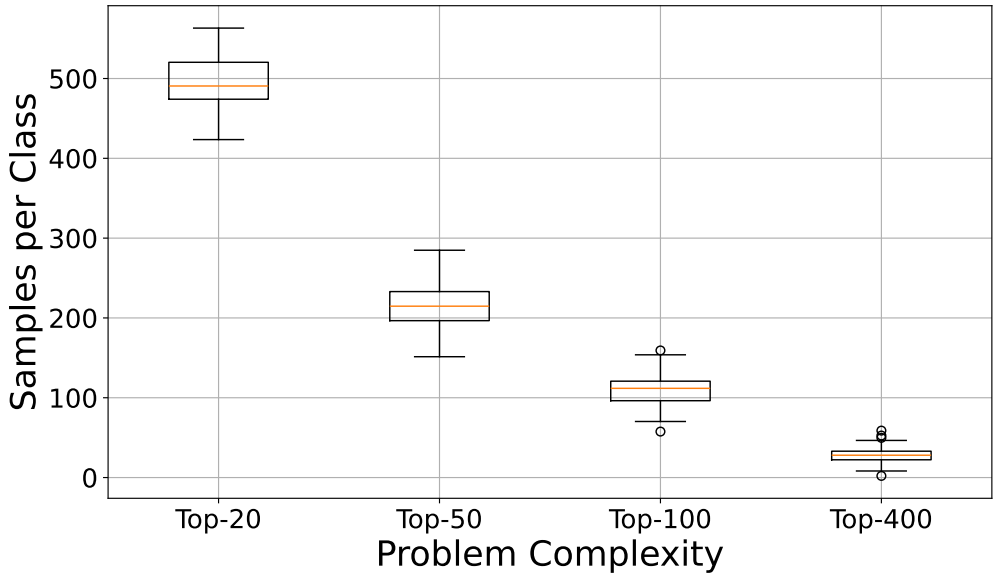


Figure 4.8: *Reduction in the number of samples per class as the complexity of the classification problem increases.*

5 Conclusion and Future Work

5.1. Conclusion

This work has presented a comprehensive study on the analysis of web browsing behavior through network traffic, with a particular focus on the impact of cookie consent decisions. The main objective was to determine whether accepting or rejecting cookies significantly affects the ability to identify visited domains based solely on captured network traffic.

To achieve this, an automated data collection pipeline was developed, capable of simulating real user navigation, interacting with cookie consent banners, and capturing the resulting traffic in *.pcap* format. These captures were then processed to extract both statistical features and sequential information, such as DNS queries, server IPs, and TLS SNI values. The resulting data was transformed into structured datasets suitable for machine learning analysis.

One of the key findings of this work is that there is no significant difference between accepting and rejecting cookies in terms of domain identification performance. The results show that the model achieves similar accuracy in both scenarios, indicating that cookies are not a determining factor for identifying browsing behavior from network traffic. Instead, other elements such as communication patterns, metadata, and external service interactions play a more relevant role.

Additionally, the analysis revealed the presence of two distinct types of domains within the traffic: those that appear consistently across most sessions (elephants) and those that are specific to individual browsing scenarios (mice). This distinction proved essential for interpreting the results, as elephant domains introduce common patterns across all samples, while mice domains provide the discriminative information required to differentiate between sessions.

Another important result is the impact of problem complexity on model performance. As the number of classes increases, the classification task becomes significantly more challenging, leading to a decrease in accuracy. This behavior is explained by the reduction in samples per class, the increased similarity between domains, and the influence of shared infrastructure services. These findings highlight the limitations of the current approach when scaling to more realistic scenarios involving a large

number of domains.

Finally, the evaluation of different embedding strategies showed that not all methods are equally suitable for this type of data. The presence of highly frequent domains and large, variable vocabularies introduces challenges for traditional techniques, reinforcing the need for approaches that are robust to high-dimensional and repetitive patterns.

Overall, this work demonstrates that, although cookie consent is relevant from a privacy perspective, it does not significantly impact the identifiability of domains from network traffic. This suggests that privacy risks extend beyond cookie management and are strongly influenced by other sources of information inherent to web communication.

5.2. Future work

Several directions can be explored to extend and strengthen the results obtained in this work, both at the level of data collection and at the level of modeling and representation.

A first line of future work concerns the automation of cookie banner detection and interaction. Although the current approach relies on rule-based mechanisms, more advanced strategies could significantly improve robustness and coverage. In particular, agentic AI approaches, combined with modern web browser automation frameworks, could be employed to dynamically analyze page structure and behavior. By reasoning over the Document Object Model (DOM), visual elements, and interaction flows, intelligent agents could adapt their actions to diverse and evolving banner designs. Additionally, the integration of Model Control Protocols (MCP) for web browsers would allow a tighter coupling between perception, decision-making, and action, reducing the number of failed or ambiguous interactions (e.g., *No Action* cases) and increasing the consistency of the collected datasets.

Another important direction relates to the design of more expressive embeddings tailored to network traffic data. While hashing-based vectorization provides a scalable and efficient solution, it treats all tokens uniformly and does not account for their differing semantic roles. Future work could explore embeddings that explicitly model the elephant–mice nature of web domains. In this context, elephant domains refer to highly frequent and persistent domains (such as content delivery networks, analytics services, or global infrastructure providers) that generate large volumes of traffic and appear across most browsing sessions, independently of the visited website. In contrast, mice domains are sparsely observed, session-specific, and directly linked to the particular website being accessed. Designing embeddings that down-weight or separately model elephant domains while emphasizing the presence and patterns of mice domains could improve discrimination between browsing sessions and reduce the dominance of generic infrastructure traffic in the feature space.

Future research could also investigate alternative representation strategies that exploit temporal

structure more explicitly. Sequence-aware models or hybrid approaches combining flow-based aggregation with sequential embeddings may capture finer-grained browsing dynamics, such as the order and timing of third-party requests relative to user interactions. These representations could be particularly valuable for distinguishing subtle behavioral differences induced by cookie consent decisions.

Regarding learning models, although this work employs a Random Forest classifier due to its robustness and interpretability, future studies could evaluate more advanced machine learning techniques. Gradient boosting methods or neural network architectures may be better suited to capture complex, non-linear relationships in high-dimensional embedded feature spaces. Moreover, models specifically designed for sequence or graph data could further leverage the structure inherent in network traffic.

Finally, an important limitation of the current study lies in the synthetic nature of the collected datasets. Automated browsing provides strong experimental control and reproducibility but may not fully reflect real user behavior, such as heterogeneous interaction patterns, varying dwell times, or background activity generated by user-installed extensions. Future work could therefore focus on incorporating more realistic browsing scenarios, potentially combining automated and human-driven sessions, as well as considering different devices, browser configurations, and geographic locations. Addressing these limitations would improve the ecological validity of the datasets and provide a more comprehensive evaluation of the proposed methodology.

Bibliography

[1] D. Perdices, J. E. López de Vergara, I. González, and L. de Pedro, "Web Browsing Privacy in the Deep Learning Era: Beyond VPNs and Encryption," *Computer Networks*, vol. 220, p. 109471, 2023.

[2] K. Ruth, D. Kumar, B. Wang, L. Valenta, and Z. Durumeric, "Toppling Top Lists: Evaluating the Accuracy of Popular Website Lists," in *Proceedings of the ACM Internet Measurement Conference (IMC '22)*, (Nice, France), pp. 374–387, ACM, 2022.

[3] K. Ruth, A. Fass, J. Azose, M. Pearson, E. Thomas, C. Sadowski, and Z. Durumeric, "A World Wide View of Browsing the World Wide Web," in *Proceedings of the ACM Internet Measurement Conference (IMC)*, pp. 317–334, ACM, 2022.

[4] Microsoft, "Automate Webpages - Power Automate." Microsoft Learn, 2024. Last updated on 10/23/2024.

[5] M. Oliveira, J. Yang, D. Griffiths, D. Bonnay, and J. Kulshrestha, "Browsing Behavior Exposes Identities On The Web," *Scientific Reports*, vol. 15, p. 36066, 2025.

[6] Y. S. Razooqi and A. Pekar, "VPN Traffic Analysis: A Survey on Detection and Application Identification," *IEEE Access*, vol. 13, pp. 132830–132848, 2025.

[7] M. Oliveira, J. Yang, D. Griffiths, D. Bonnay, and J. Kulshrestha, "Browsing behavior exposes identities on the Web," *arXiv preprint arXiv:2312.15489*, 2023.

[8] SmartSurvey, "How We Use Cookies," 2025.

[9] SurveyMonkey, "Cookies Used on Survey Pages," 2025.

[10] A. Abdallah, A. Ahmad, and B. Said *Journal of Open Innovation: Technology, Market, and Complexity*, vol. 11, p. 100520, 2025.

[11] Secra Solutions, "What is a JWT (JSON Web Token): How it works, Vulnerabilities and Best Practices," May 2026.

[12] N. Namer, "Analysis of Compromised User Authentication Threats for JSON Web Tokens," June 2023.

[13] Microsoft, "Microsoft Identity Platform and OAuth 2.0 Authorization Code Flow," 2026.

APPENDICES

A

Alternative Client-Side Storage Mechanisms

Although cookies are one of the most widely known mechanisms for storing information on the client side, modern web applications rely on several additional storage technologies to persist data in the browser. These mechanisms allow websites to improve user experience, maintain sessions, cache information, and support personalization features. However, they also introduce important privacy implications, as they may be used for tracking and behavioral profiling purposes even when traditional cookies are restricted or rejected.

A.1. Local Storage

Local Storage is part of the Web Storage API and allows websites to store persistent key-value data directly in the browser. Unlike cookies, Local Storage data is not automatically transmitted with every HTTP request, which reduces network overhead and allows significantly larger storage capacities, typically up to several megabytes per domain.

Data stored in Local Storage remains available even after the browser is closed and reopened, unless the user manually clears the browser storage. Web applications commonly use Local Storage to save user preferences, interface configurations, authentication tokens, and cached application data.

From a privacy perspective, Local Storage can also be used for tracking purposes. Since its contents persist across browsing sessions, websites and third-party scripts may exploit Local Storage identifiers similarly to persistent cookies in order to recognize returning users.

A.2. Session Storage

Session Storage is another mechanism provided by the Web Storage API. It operates similarly to Local Storage by storing key-value pairs within the browser, but its lifetime is limited to the duration of the current browser tab or session.

Once the tab or browser window is closed, the stored data is automatically removed. Session Stora-

ge is therefore commonly used for temporary session-related information such as form data, navigation state, or temporary authentication variables.

Compared to Local Storage, Session Storage provides a more limited persistence model and is generally considered less intrusive from a privacy perspective. Nevertheless, it can still contribute to user tracking during active browsing sessions.

A.3. IndexedDB

IndexedDB is a low-level client-side database system integrated into modern browsers. Unlike cookies or Web Storage, IndexedDB supports structured and large-scale data storage, allowing websites to store complex objects, binary data, and offline application resources.

Modern web applications frequently use IndexedDB to implement offline functionality, synchronize cached content, and improve application performance. Due to its large storage capacity and persistence, IndexedDB has become an important component in Progressive Web Applications (PWAs).

From a privacy standpoint, IndexedDB may also be leveraged to store long-term identifiers and behavioral information. Since its storage persists across sessions and is less visible to users than cookies, it represents another potential vector for persistent tracking mechanisms.

A.4. Privacy Implications

The existence of multiple client-side storage mechanisms demonstrates that modern web tracking extends far beyond traditional HTTP cookies. Even when users reject cookies or configure privacy protections in their browsers, websites may still rely on Local Storage, Session Storage, IndexedDB, browser caches, and other persistent technologies to store identifiers and monitor browsing behavior.

These mechanisms are frequently combined with analytics services, advertising platforms, and browser fingerprinting techniques in order to build detailed behavioral profiles of users across browsing sessions. Since many of these technologies operate in the background and are less visible than traditional cookies, users are often unaware of the amount of information being collected and stored locally by websites.

As a consequence, modern privacy protection cannot rely exclusively on restricting cookies. Current web ecosystems involve multiple layers of storage and tracking technologies that together contribute to user identification and behavioral analysis.

B

Accept Only Essentials Consent Configurations

In addition to the traditional *Accept* and *Reject* consent options, many modern websites include intermediate configurations such as *Accept Only Essentials*, *Necessary Cookies Only*, or similar alternatives within cookie consent banners. These options are designed to allow the use of strictly necessary cookies required for the correct operation of the website while rejecting optional tracking, analytics, advertising, or personalization technologies.

Essential cookies typically include session management, authentication mechanisms, security protections, language preferences, and other functionalities required for the proper delivery of the service requested by the user. Since these cookies are generally considered technically necessary, many privacy regulations such as the GDPR and the ePrivacy Directive allow their usage without requiring explicit consent.

From a traffic analysis perspective, the behavior generated under *Accept Only Essentials* configurations may partially resemble the *Deny* scenario, since many third-party trackers, analytics services, and advertising-related domains are disabled. However, it cannot always be assumed that both scenarios are identical. Depending on the implementation of the consent management platform and the internal architecture of the website, some analytics or background services may still remain active even when only essential cookies are accepted.

As a consequence, *Accept Only Essentials* configurations often represent an intermediate state between full acceptance and full rejection of cookies. However, as observed throughout the experimental results of this work, the differences in classification accuracy between consent scenarios are relatively limited. Therefore, it is reasonable to expect that traffic generated under *Accept Only Essentials* configurations would produce accuracy values and traffic patterns relatively similar to those obtained in the *Accept* and *Deny* scenarios, although small variations in domain distributions and tracking-related requests could still be present depending on the implementation of each website.


UÁM
Universidad Autónoma
de Madrid